A Behavioral Model for Reality-Gap Mitigation in Digital Twins of Omnidirectional Soccer Robots

REPLACE-ME First Author*, REPLACE-ME Co-author*
*Centro de Informática, Universidade Federal de Pernambuco (CIn-UFPE), Recife, Brazil
RobôCIn Robotics Team
Email: {REPLACE-ME}@cin.ufpe.br

***Abstract*—Teams in the RoboCup Small Size League develop and validate control software largely in simulation, where simplified physics leaves a reality gap to the field. This paper asks which behavioral model of an omnidirectional soccer robot can be identified from the competition logs teams already record, and what those logs cannot supply. Each body-frame degree of freedom is modeled as a first-order-plus-dead-time channel. The two translational channels are identified from in-game logs and reach roughly 87 percent hold-out fit. The yaw channel is not: during play the yaw command carries a median of one percent of the per-wheel actuation and explains only 56 percent of the measured rotation, so a short dedicated open-loop campaign restores the excitation and identifies the channel at 71 percent hold-out fit. Fidelity is evaluated in free run over horizons of 0.1 to 1.5 seconds, with every plant in the same harness. Over 73.5 seconds of held-out motion from ten events, the model reaches a median position error of 0.039 meters at the 0.4-second horizon, against 0.101 for ideal kinematics and 0.129 for rSim, the ODE engine of the rSoccer framework, run untuned and reported as context. The model is lower in all ten events, and an order-independent ablation attributes the gain in comparable parts to the time constant and the dead time, conditional on the measured initial velocity. Because 99.6 percent of the hold-out windows are translation-dominated, rotational fidelity remains unvalidated; a screening computable from the logs alone predicts which channels a log can identify.**

***Index Terms*—digital twin, system identification, robot modeling, omnidirectional robot, robot soccer**

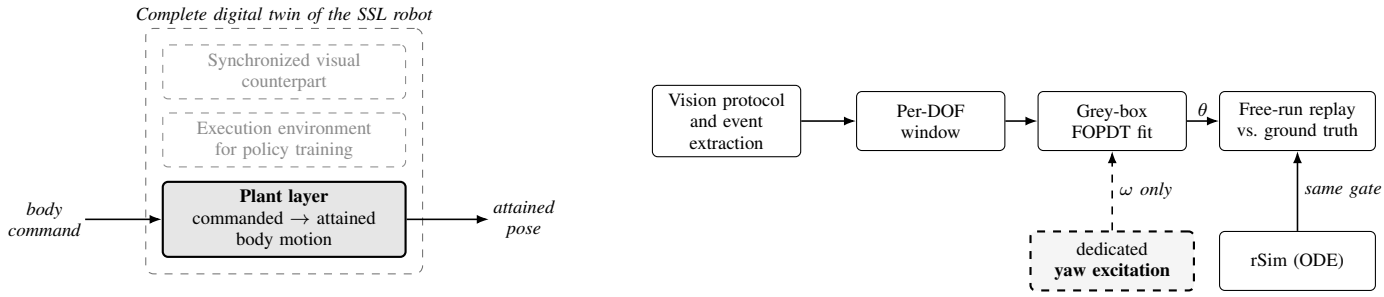
I. Introduction

The RoboCup Small Size League (SSL) fields teams of fast, omnidirectional, four-wheeled robots whose coordination software is written, tuned, and regression-tested almost entirely on simulators. The league's reference simulation tools implement a simplified omnidirectional wheel model on top of a rigid-body ODE engine [1]; reinforcement-learning frameworks inherit that engine [2]; and recent motion-planning policies for the SSL are trained in these environments [3]. Because the simulated dynamic response departs from that of the physical robot, behavior that passes validation in simulation often degrades once deployed. This discrepancy is what the literature calls the reality gap.

Recent efforts to narrow it converge on *digital twins*: simulation models bound to one specific physical asset and kept faithful to it by data measured from that asset, rather than generic models of a class of robots. Twins of mobile platforms have been built for commercial AGVs [4], and for the navigation and control of omnidirectional robots [5], and grey-box identification has been evaluated as the module that produces the model within such a twin [6]. Building one is not free: it normally presumes either a detailed physical description of the mechanism, whose mechanical parameters teams rarely have, or a dedicated excitation campaign, which demands the robot, the field, and time outside of competition.

Competition logs are an attractive alternative because teams already record them: body-frame velocity commands and vision poses at roughly 60 Hz during closed-loop play. Existing work, however, reflects the difficulty of using them by avoiding it. Simulation models of omnidirectional platforms are tuned on dedicated real trajectories [7], [8], identification pipelines design excitation sequences per parameter group [9], real-time estimation for twins assumes informative operating data [10], and offline sim-to-real calibration presumes the pre-collected data excite the parameters of interest [11]. Whether a behavioral model can be identified from closed-loop *operational* logs, and which degrees of freedom (DOF) such logs support, remains underexplored. This paper answers that question for an SSL robot modeled as three decoupled first-order-plus-dead-time (FOPDT) channels, one per DOF, evaluated in free run against a zero-parameter baseline under an identical harness.

Contributions. (C1) A per-DOF grey-box FOPDT identification methodology for an omnidirectional robot from *closed-loop* competition logs, validated on held-out events, for the degrees of freedom those logs excite. **(C2)** A fidelity protocol that scores the model the way it is used: driven only by the recorded commands from a known pose, with no correction from measured data inside the window. Verdicts are issued per motion regime and only when a paired test, an effect size, and a bootstrap interval agree. **(C3)** A translational fidelity result with the improvement attributed to its source: the model lowers the median position error of the baseline in all ten hold-out events, and an order-independent ablation splits the effect between the time constant and the dead time, conditional on the measured initial state. **(C4)** A pre-fit screening procedure that predicts, from an operational log alone, which degrees of freedom that log can identify. It combines two quantities computable before any model is estimated: the share of the actuation each command carries and the variance of the attained velocity it explains. On this robot it separates the identifiable channels from the unidentifiable one before a single parameter is fitted.



(a) Scope of the paper **(b) Identification and evaluation pipeline**

Fig. 1. (a) The paper delivers the plant layer of a larger digital twin: the map from commanded to attained body motion. (b) Solid path: closed-loop competition logs. Dashed branch: the dedicated open-loop collection, which feeds the ω channel alone. Every comparison plant is scored on the same windows, horizons and gates as the model.

II. Related Work

A. Behavioral twins of mobile robots

Digital twins of mobile platforms are typically validated behaviorally, by comparing simulated and real trajectories rather than parameters [4], [5]. On the construction side, grey-box identification has been assessed as the model-generating module of a twin and has been found to be sensitive to the informativeness of its input data [6]. Work on omnidirectional platforms assumes dedicated experiments, whether one experiment per parameter group [9] or tuning against motion-capture trajectories, manually and per trajectory [7], or by black-box optimization over a few high-impact parameters [8]. The per-DOF FOPDT structure adopted here is the classic first-order-plus-dead-time process model of control practice [12]. The closest precedent for reusing data that already exists, rather than collecting new data, is DROPO, which estimates domain-randomization distributions from an offline, pre-collected dataset [11]; like the others, it presumes that the offline data excite the parameters being estimated.

B. SSL simulators and league context

The SSL's reference simulator, grSim, adopts a simplified dynamic model of the omni wheels for real-time performance [1]; the rSoccer framework derives its rSim engine from grSim for reinforcement-learning workloads [2], and planning policies are trained in such environments [3]. A Simscape multibody (white-box) twin of an SSL robot reports trajectory RMSE of 150–240 mm on in-game scenarios with large horizons [13]. League practice routinely records vision poses, odometry, and onboard velocities [14], and SSL firmware converts body commands into per-wheel velocity loops [15].

C. Closed-loop identification

That feedback correlates the input with unmeasured disturbances, breaking methods that work in open loop, is classic [16]–[18], as is the role of persistent excitation in prediction-error estimation [19].

III. Method

This paper specifies the *plant layer* of a larger digital-twin effort: a complete twin of an SSL robot also needs a synchronized visual counterpart and an execution environment in which policies can be trained, but the component that governs whether such an environment reproduces the robot's behavior at all is the map from commanded to attained body motion (Fig. 1a).

The pipeline runs from the league protocol stream through event extraction and per-DOF window labeling to a grey-box FOPDT fit per channel, and the resulting nine parameters drive a free-run replay scored against ground truth (Fig. 1b). Competition logs feed that path end to end. Every comparison plant is scored on the same windows, horizons, and statistical gates as the model.

A. Per-DOF FOPDT twin

We model the robot as three decoupled single-input single-output channels, one per degree of freedom (DOF) $i \in \{v_x, v_y, \omega\}$, each a first-order-plus-dead-time (FOPDT) transfer function from commanded to attained body velocity,

$$G_i(s) \;=\; \frac{K_i\, e^{-T_{di}s}}{\tau_i s + 1}, \tag{1}$$

with per-DOF gain K_i, time constant τ_i, and transport delay T_{di}. The inputs are the body-frame velocity commands and the outputs are the attained body velocities. The full parameter vector has fixed order $[K_{vx}, \tau_{vx}, T_{dvx}, K_{vy}, \tau_{vy}, T_{dvy}, K_\omega, \tau_\omega, T_{d\omega}]$ (nine parameters). This is a *behavioral* (observational) model: it targets the input-to-pose response, not a white-box description of wheel/contact mechanics.

B. Discretization and pose integration

Identification and replay both run on a uniform grid with a step size of $\Delta t = 1/60\,\mathrm{s}$, the nominal vision rate. Each DOF's command is delayed by a zero-order-hold (ZOH) integer shift of $m_i = \lceil T_{di}/\Delta t \rceil$ samples (zero before the segment start), passed through the first-order update $v_i[k] = v_i[k-1] +$

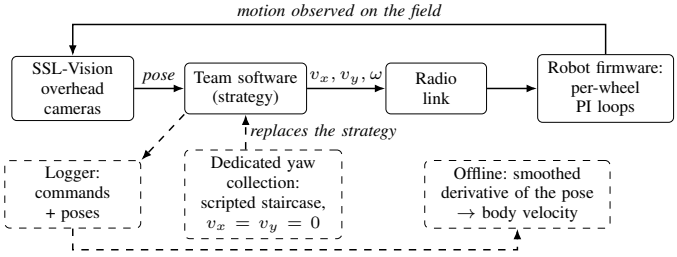


Fig. 2. Measurement chain: the league vision system reports the field-frame pose at about 60 Hz; the team software closes the loop and sends body-frame velocity commands over the radio to per-wheel loops on the robot; a logger records the commands and the poses, which is all the identification consumes. Body velocities are derived offline from the vision pose (Sec. III-C). For the dedicated yaw collection a scripted staircase of yaw commands replaces the strategy, open loop.

$\alpha_i(u_i^d[k] - v_i[k-1])$, and scaled to the attained body velocity $\hat{v}_i[k] = K_i\, v_i[k]$ (instantaneous for $\tau_i \to 0$).

C. Data and pre-processing

The data are closed-loop competition logs of a single omnidirectional 4-wheeled SSL robot, decoded from the league protocol stream into per-event body-frame commands and field-frame pose at $\sim$60 Hz. Measured body velocity, used only for identification and for the initial condition, is obtained by smoothing the vision position with a low-order polynomial (Savitzky–Golay) filter, differentiating it by central differences, and rotating the result into the body frame. Fig. 2 shows the measurement chain: the body velocity used for identification is not read from the robot's wheel encoders but derived offline from the vision pose, both for the competition logs and for the dedicated yaw collection.[1]

Logs arrive on an irregular time grid, since commands are transmitted in bursts: identification resamples every signal onto the uniform $1/60$ s grid, whereas the fidelity metric leaves the samples where they were recorded and defines its windows in seconds, so that no interpolation enters the reported error. The dataset comprises 46 events across four motion labels (`front_to_back`, `side_to_side`, `shoot_to_goal`, `square`); a frozen list of 10 events is held out and was not used for fitting.

Throughout, *fit* measures how much of the variation in the measured signal the model reproduces, on events the fit never saw:

$$\text{fit} = 100\left(1 - \frac{\|y - \hat{y}\|_2}{\|y - \bar{y}\|_2}\right)\ [\%], \tag{2}$$

where y is the measured body velocity of the degree of freedom being identified, $\hat{y}$ is the model's output when driven by the commands alone, and $\bar{y}$ is the mean of y.

D. Identification

Each translational DOF is identified independently as a P1D (FOPDT) model by prediction-error minimization, with an event-level hold-out. The excitation diagnostic selects the cleanest source label per DOF: v_x from `front_to_back` and v_y from `side_to_side` (command-to-velocity cross-correlation ≈ 0.99). Yaw is instead identified from a dedicated open-loop collection, with the same model structure and the same estimator, because the competition logs carry too little yaw excitation to support the fit. The dedicated dataset is single-session and single-robot with $v_x = v_y = 0$: an alternating-sign staircase of commanded yaw rates $\{0.25, 0.5, 1, 2.5, 5\}$ rad/s, ~ 10 s per step.

E. Frames, constants and replication

Commands and attained velocities live in the *body* frame: v_x longitudinal, v_y lateral, ω yaw rate about the upward z axis, counter-clockwise positive. Pose is integrated in the *field* frame as $\dot{X} = v_x \cos\theta - v_y \sin\theta$, $\dot{Y} = v_x \sin\theta + v_y \cos\theta$, with θ unwrapped. Per-wheel geometry converts the firmware's per-wheel motor threshold (1.567 rad/s) into a body yaw rate for Sec. IV-D, through the ratio of the center-to-wheel-contact radius $R = 0.0796$ m to the wheel radius $r = 0.0295$ m.

F. Free-run replay and multi-horizon metric

Fidelity is measured in *free-run*: segments are cut into overlapping windows at a fixed stride, each re-anchored to the true $\mathrm{SE}(2)$ pose at its anchor t_0 and rolled out to $H_{\max} = 90$ samples (1.5 s) driven only by the command stream. The Euclidean position error $e_{\text{pos}}(w, h)$ and the wrapped absolute heading error $e_{\text{ang}}(w, h)$ at $h \in \{6, 12, 24, 42, 60, 90\}$ samples ($\{0.1, 0.2, 0.4, 0.7, 1.0, 1.5\}$ s), with reference horizon $h_{\text{ref}} = 24$ (0.4 s) are reported separately. e_{pos} is the claim of this paper; e_{ang} is reported but bounded by the population of rotation-rich windows available (Sec. IV-B).

G. Baselines

We compare the twin against two baselines under the *same* windows, horizons, and gate. The first is an *ideal-kinematics* plant in which the attained body velocity equals the command instantaneously (no gain, lag, or delay), integrated by the same trapezoidal scheme. The second is **rSim**, the ODE-based physics engine of the rSoccer environment, run with its stock robot parameters and no fitting to these logs, so the comparison is between a plant fitted to this robot and an untuned engine; it is reported as context, not as the primary comparison. The recorded body-frame commands enter it directly as body-velocity setpoints, with no motion controller in between, and it is stepped in 1 ms substeps interpolated back onto the $1/60$ s grid. rSim has no velocity setter at reset, so each window warms it up over the 15 real pre-anchor commands before a rigid $\mathrm{SE}(2)$ re-anchor at t_0; the dead-time term of the model has no counterpart in it.

IV. Results

A. Identified model

Table I reports the per-DOF parameters. Both translational channels identify as fast FOPDT models, with hold-out fit 86.7% (v_x) and 87.1% (v_y); against the previous, slower parameters, they improve every event-by-horizon cell of the

[1] Supplementary videos (the dedicated yaw collection with the velocity extraction, and the model driven by the team's strategy): https://REPLACE-ME.

TABLE I

IDENTIFIED PER-DOF FOPDT PARAMETERS AND HOLD-OUT FIT. ALL THREE CHANNELS ARE IDENTIFIED (NO PLACEHOLDER); ω USES A DEDICATED OPEN-LOOP ROTATION-ONLY COLLECTION (SEE SEC. IV-D), NOT THE CLOSED-LOOP COMPETITION LOGS USED FOR v_x, v_y.

DOF	K	τ (s)	T_d (s)	hold-out fit
v_x	1.119	0.072	0.069	86.7 %
v_y	1.052	0.127	0.056	87.1 %
ω	1.002	0.083	0.089	71.1 %

Fit is Eq. 2, averaged over hold-out events. Because v_x has a single hold-out event, we also ran leave-one-event-out over all events of each translational label: v_x gives 84.3 ± 2.0 p.p. over 4 folds and v_y 82.8 ± 2.7 p.p. over 13, with the refitted gain within 0.015 of the promoted value in both cases. No parameter intervals are reported: see Sec. IV-D.

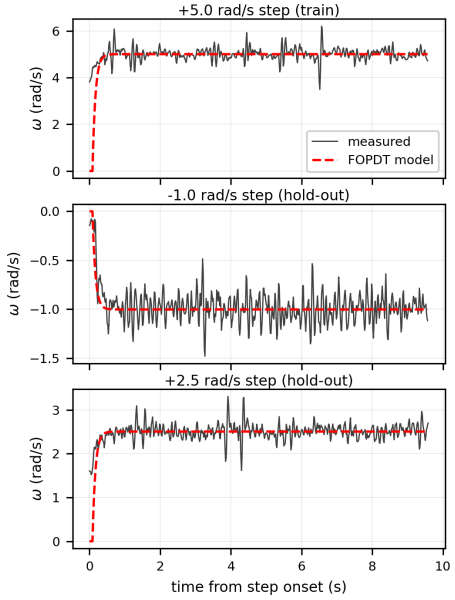


Fig. 3. Measured vs. simulated yaw rate on the dedicated open-loop staircase (one training, two hold-out steps). Same model structure and identification tooling as on the competition logs; only the excitation regime differs.

SE(2) gate. Yaw is identified from a separate open-loop dataset at 71.1%.

B. Motion distribution of the hold-out

The hold-out windows are overwhelmingly translation-dominated: of 265 dense windows (49 non-overlapping), 264 fall in the translation bin (99.6%), one is rotation-dominated, and none are mixed. Re-classifying after ω was identified reproduces the same $264/1/0$ split and leaves both other bins with $N_{\text{indep}} = 0$, now for lack of rotation-rich competition windows rather than for lack of a yaw model.

C. Translational fidelity

Ideal kinematics is the primary comparator: both plants run in the same harness, start from the same measured body velocity and are scored on the same windows, so the model is the only difference. Its median position error is lower at every horizon (Fig. 4, Table II): 0.039 m against 0.101 m at h_{ref}, and 0.119 m against 0.291 m at 1.5 s. For scale, the robot covers about 0.4 m within h_{ref} and its contact diameter is 0.16 m: the baseline misses by two thirds of a robot, the model by a quarter.

TABLE II

MEDIAN FREE-RUN ERROR ON TRANSLATION-DOMINATED HOLD-OUT WINDOWS. POSITION ERROR IN METERS, HEADING ERROR IN RADIANS. THE MODEL IS LOWEST IN BOTH QUANTITIES AT EVERY HORIZON; THE TWO BASELINES CROSS IN POSITION NEAR 1 s.

	e_{pos} (m)			e_{ang} (rad)		
h (s)	model	ideal	rSim	model	ideal	rSim
0.10	**0.008**	0.026	0.057	**0.013**	0.023	0.026
0.20	**0.018**	0.050	0.092	**0.031**	0.044	0.046
0.40	**0.039**	0.101	0.129	**0.044**	0.067	0.064
0.70	**0.062**	0.152	0.174	**0.063**	0.076	0.079
1.00	**0.092**	0.217	0.213	**0.073**	0.092	0.086
1.50	**0.119**	0.291	0.289	**0.090**	0.095	0.094

The hold-out is ten events, 73.5 s of scored motion. Windows from one event share robot, surface, session and controller, so we take the *event* as the unit of analysis. The model wins all ten, by a median of +0.069 m (one-sided Wilcoxon $p = 9.8 \times 10^{-4}$, the smallest value attainable at $n = 10$); the pre-registered window-level test over the 49 non-overlapping windows agrees and is the less conservative of the two.

1) What produces the improvement: We switch each of $\{K, \tau, T_d\}$ off in every combination and average over all orderings, so no term is credited merely for being enabled first. Of the 0.062 m separating the two plants at h_{ref}, the time constant accounts for 55%, the dead time for 43% and the gain for 2%. Lag and delay are comparable contributors; the gain is irrelevant.

2) The role of the measured initial velocity: Both plants are seeded with the measured body velocity, but only the dynamic one can carry it forward, and neither ingredient works alone. Extrapolating that velocity at constant speed, ignoring the commands, gives 0.160 m; running the model from rest gives 0.114 m. Both are worse than ideal kinematics, and only the combination reaches 0.039 m. We treat the measured velocity as part of the model interface rather than privileged information, since a twin running alongside the robot has that estimate; re-deriving it with a strictly causal difference raises both errors but *widens* the gap, so the verdict does not rest on it.

3) The untuned ODE simulator: rSim reaches 0.129 m at h_{ref}, so even the zero-parameter baseline outperforms it. Giving it the model's dead time widens rather than closes the gap.

4) Heading error: The rotation and mixed bins lack a scorable population (Sec. IV-B), so heading carries no verdict. Descriptively (Table II), the model's median e_{ang} is the lowest of the three plants at every horizon, but the margin shrinks from roughly half the baselines' at 0.1 s to within 0.005 rad of both by 1.5 s. The position advantage does not transfer to heading in the same proportion, which is consistent with yaw being the channel the competition logs cannot constrain.

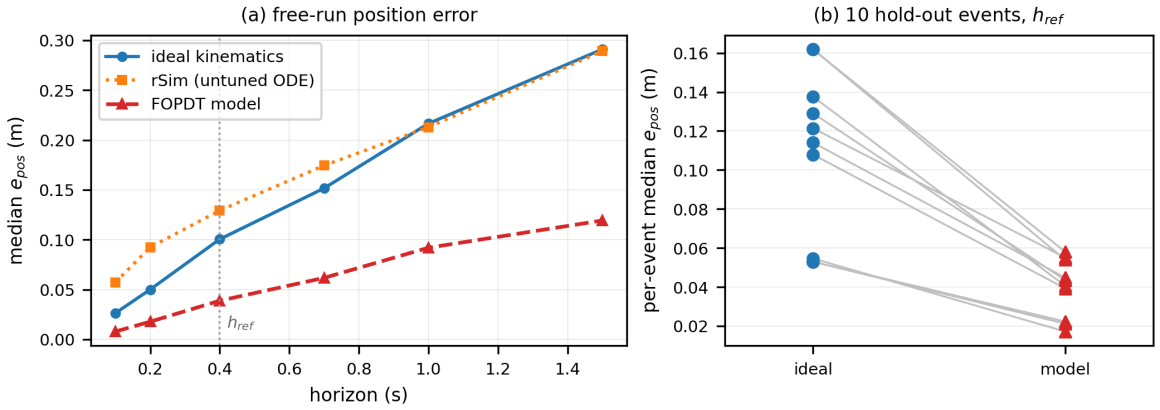


Fig. 4. (a) Median free-run position error against horizon on translation-dominated hold-out windows, for the FOPDT model, ideal kinematics, and an untuned rSim. The two baselines cross near 1 s. (b) The same comparison resolved by event at h_{ref}: each line is one of the ten hold-out events, and the model is lower in all ten.

D. Yaw identifiability is a function of excitation regime

The problem is visible in the logs before any model is fitted. Regressing the measured body velocity on the command, aligned by its dead time, the command explains 96% of the variance for v_x and 94% for v_y, but only 56% for ω: nearly half the rotation the cameras see in play is not attributable to the yaw command.

Fitting the same structure to the highest-rotation competition events gives $K_\omega = 1.378$ and a hold-out fit of 44.8%, below the pre-registered 60% gate. Using it also *worsens* free-run position, because a gain above unity amplifies accumulated heading drift, so it is not promoted.

1) Why the logs cannot constrain yaw: The firmware clamps each *wheel* setpoint below $1.567\,\mathrm{rad/s}$, after the command kinematics. Converted to body yaw rate that threshold sits at 0.58–$0.60\,\mathrm{rad/s}$, and 94.3% of commanded yaw samples in the logs fall below it. The statistic is easy to misread: it does not mean yaw is switched off. In play the wheels are already turning because of translation, so among those samples the median per-wheel translational setpoint is $15\times$ the clamp and only 2.2% are fully clamped. The yaw command is not suppressed but *diluted*, contributing a median of 1.0% of what each wheel is asked to do. What the actuators receive in competition is 99% translation.

2) The same structure on dedicated data: A short dedicated open-loop collection removes the dilution: with $v_x = v_y = 0$ the yaw command becomes the sole actuation. Identified with the *same* structure and tooling (Fig. 3), it gives $K_\omega = 1.002$, $\tau_\omega = 0.083\,\mathrm{s}$, $T_{d\omega} = 0.089\,\mathrm{s}$ and a hold-out fit of 71.1%, above the gate. Scored against the same fixed placeholder yaw channel ($K = 1$, $\tau = 0.1\,\mathrm{s}$, $T_d = 0$) on the same competition population, the closed-loop estimate is worse at every horizon and the open-loop one better, so the verdict inverts under identical tooling.

V. Discussion

A. Identifiability is a property of the data, not the DOF

The central finding is not that yaw is harder to model than translation, but that the *same* structure and tooling give opposite verdicts depending on the excitation regime: on closed-loop logs the yaw P1D meets none of the acceptance criteria, while on a small dedicated collection it passes every pre-registered gate with near-unity gain. Nothing in the pipeline distinguishes the DOFs; only the data does, since those logs excite v_x, v_y persistently while the yaw command is a median 1% of the per-wheel actuation.

The closed-loop gain estimate is not merely imprecise; it is wrong in the same direction across two independent attempts, a direct high-rotation fit and a regime-restricted linear fit. The dilution result of Sec. IV-D says what that estimator is actually fitting: with the yaw command at about 1% of the per-wheel setpoint, it explains only 56% of the variance of the measured rotation, the remainder plausibly coming from slip, contact and traction asymmetry generated by the translational motion. Regressing measured yaw on commanded yaw is then dominated by whatever correlates with the command, and under feedback the controller issues yaw precisely in response to those disturbances, which is the textbook closed-loop identification hazard [16]–[18]. This removes the reading that the actuation threshold simply switches yaw off, since it engages in only $\sim 2\%$ of samples, and replaces it with something measurable. It does not isolate feedback from dilution, because the dedicated collection removes both at once.

B. Limitations

The yaw fit rests on six events from a single session, with no fine amplitude sweep, so its threshold is an interval; all data come from one robot, one surface and one hold-out split, and pre-registration and deterministic re-execution mitigate but do not replace replication. The two yaw campaigns differ in loop topology, yaw-to-translation ratio and session at once, so

separating the last two would need an open-loop arm with translation superimposed, which we did not collect; nor did we attempt the closed-loop-aware estimators (instrumental-variable, two-stage, joint input–output) the cited literature proposes. Ljung–Box rejects residual whiteness for both yaw fits, which the smoothing differentiator explains and which is why no parameter intervals are reported. The firmware and CAD radius ratios still differ by $\sim 3.3\%$. On evaluation, 99.6% of hold-out windows are translation-dominated, so rotational fidelity is not validated; the heading advantage all but vanishes by $1.5\,\mathrm{s}$ (Table II), so the position claim does not transfer to heading in the same proportion; and the rSim comparison remains fitted-versus-untuned and confounded by initial state, which the matched-delay ablation narrows but does not remove. The model is also deliberately simple, and three simplifications bound where it applies. The channels are decoupled: no term maps a translational command into yaw or a yaw command into translation, although part of the 44% of the measured rotation that the yaw command leaves unexplained in play is plausibly translation-induced yaw (slip, contact and traction asymmetry), which is exactly what a coupled model would have to capture; a coupled (MIMO) identification on the same data is the natural next step and is not attempted here. The plant is linear, with no saturation, acceleration limit or dead zone: it is fitted and scored inside the envelope of the logs, where the commanded speed exceeds $2.9\,\mathrm{m/s}$ in only 0.01% of the competition samples and the dedicated yaw steps peak at about a quarter of the per-wheel PWM limit, so its validity in aggressive or mixed-motion regimes is unverified. Slip enters only through the identified gains and lags, never as a state.

VI. Conclusion and Future Work

This paper presented a per-DOF FOPDT model of an omnidirectional SSL robot, identified mostly from closed-loop competition logs, which attains validated translational fidelity: over $73.5\,\mathrm{s}$ of scored hold-out motion it lowers the median position error of a baseline in all ten hold-out events, halving it at the reference horizon, with the improvement attributable in comparable parts to the time constant and the dead time, and only in combination with the measured initial velocity. Yaw needed a short dedicated open-loop campaign, after characterizing why the logs cannot support it: during play the yaw command carries a median 1% of the per-wheel actuation and explains only 56% of the variance of the measured rotation. Code, configurations, and both datasets are released at https://REPLACE-ME, with instructions to reproduce every number above. Future work embeds this plant in a full digital twin inside a robot-learning environment such as Isaac Sim or MuJoCo [3], and adds a rotation-rich validation distribution, an open-loop arm with translation superimposed, and cross-robot replication.

Acknowledgment

REPLACE-ME: funding agencies and grant numbers; the RobôCIn team; who ran the dedicated yaw collection and who produced the CAD model.

References

[1] V. Monajjemi, A. Koochakzadeh, and S. S. Ghidary, "grSim – RoboCup small size robot soccer simulator," in *RoboCup 2011: Robot Soccer World Cup XV, LNCS 7416*. Springer, 2012, pp. 450–460.

[2] F. B. Martins, M. G. Machado, H. F. Bassani, P. H. M. Braga, and E. S. Barros, "rSoccer: A framework for studying reinforcement learning in small and very small size robot soccer," arXiv:2106.12895, 2021.

[3] M. G. Machado, J. G. Melo, C. Zanchettin, P. H. Braga, P. V. Cunha, E. N. S. Barros, and H. F. Bassani, "Planning the path with reinforcement learning: Optimal robot motion planning in RoboCup Small Size League environments," Preprint.

[4] W. E. Barioni, I. P. Latini, C. T. S. Barioni, F. Neves-Jr, L. V. R. Arruda, and M. Teixeira, "Development and validation of a digital twin for a commercial mobile robot," in *Proc. IEEE Int. Conf. Robot. Comput. (IRC)*, 2024.

[5] H. Yang, Z. Qin, Y. Xia, and F. Cheng, "Digital twin-based autonomous navigation and control of omnidirectional mobile robots," *IEEE Trans. Veh. Technol.*, vol. 74, no. 4, pp. 5687–5697, 2025.

[6] R. W. Brennan and J. Lesage, "Evaluating the use of grey-box system identification for digital twins in manufacturing automation," *Int. J. Comput. Integr. Manuf.*, vol. 38, no. 7, pp. 893–914, 2025.

[7] M. Wiedemann, O. Ahmed, A. Dieckhöfer, R. Gasoto, and S. Kerner, "Simulation modeling of highly dynamic omnidirectional mobile robots based on real-world data," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Yokohama, Japan, 2024.

[8] M. Wiedemann, O. Ahmed, M. Hatwar, R. Gasoto, P. Detzner, and S. Kerner, "Automated tuning of non-differentiable rigid body simulation models for wheeled mobile robots," in *Proc. IEEE Int. Conf. Autom. Sci. Eng. (CASE)*, Los Angeles, CA, USA, 2025.

[9] P. Giró, E. Celaya, and L. Ros, "The Otbot project: Dynamic modelling, parameter identification and motion control of an omnidirectional tire-wheeled robot," arXiv:2311.10834, 2023.

[10] S. R. Kosmaga, A. Widyotriatmo, E. Rijanto, and A. Hasan, "Digital twin-enabled real-time parameter estimation of a wheeled mobile robot," *Syst. Sci. Control Eng.*, vol. 14, no. 1, p. 2634459, 2026.

[11] G. Tiboni, K. Arndt, and V. Kyrki, "DROPO: Sim-to-real transfer with offline domain randomization," *Robot. Auton. Syst.*, vol. 166, p. 104432, 2023.

[12] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Instrument Society of America, 1995.

[13] V. Araújo and E. Barros, "Digital twin-based architecture and design for co-simulation of omnidirectional mobile robots," in *Proc. Latin Amer. Robot. Symp. (LARS)*, 2025.

[14] J. G. Melo, L. Cavalcanti, R. Joaquim, V. Araújo, and E. Barros, "Dataset and baseline experiments for self-localization and tracking in the RoboCup Small Size League," in *Proc. LARS/SBR/WRE*, 2023.

[15] A. Freiberger, M. R. O. A. Maximo, and D. Loubach, "Implementation of firmware based on real-time operating system for Small Size League robot," in *Proc. Brazilian Symp. Robot. (SBR/WRE)*, 2024.

[16] I. Gustavsson, L. Ljung, and T. Söderström, "Identification of processes in closed loop—identifiability and accuracy aspects," *Automatica*, vol. 13, pp. 59–75, 1977.

[17] P. Van den Hof, "Closed-loop issues in system identification," *Annu. Rev. Control*, vol. 22, pp. 173–186, 1998.

[18] U. Forssell and L. Ljung, "Closed-loop identification revisited," *Automatica*, vol. 35, pp. 1215–1241, 1999.

[19] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Prentice Hall PTR, 1999.